Week No. 3

Lab Manual: Information Security

Lab Week 3: Network Traffic

with Wireshark Lab

Objectives

- Capture, filter, and analyze live network traffic using Wireshark.
- Understand ICMP, TCP, and application-layer packets.
- Learn how filtering helps isolate relevant traffic.
- Analyze TCP 3-way handshake and packet metadata.

Lab Overview

This lab focuses on hands-on network monitoring and analysis using Wireshark. Students will capture ICMP/HTTP packets, apply filters, decode TCP handshakes, and analyze traffic behavior.

Experiment 1 – Capturing ICMP Packets

(Ping) Theory

ICMP is used for diagnostic testing and connectivity checks (ping). When you ping a host, ICMP Echo Request and Echo Reply packets are generated.

Steps

Step 1 — Launch Wireshark

- Open Wireshark
- Select appropriate network interface:
    - eth0 (VM bridged)
    - wlan0 (Wi-Fi)
    - lo (loopback)

Step 2 — Start Capture

Click **Start Capturing Packets.**

Step 3 — Generate ICMP Traffic

Open terminal:

ping -n 5 google.com / -c



Step 4 — Stop Capture

Click **Stop** in Wireshark.

Step 5 — Apply ICMP Filter

icmp

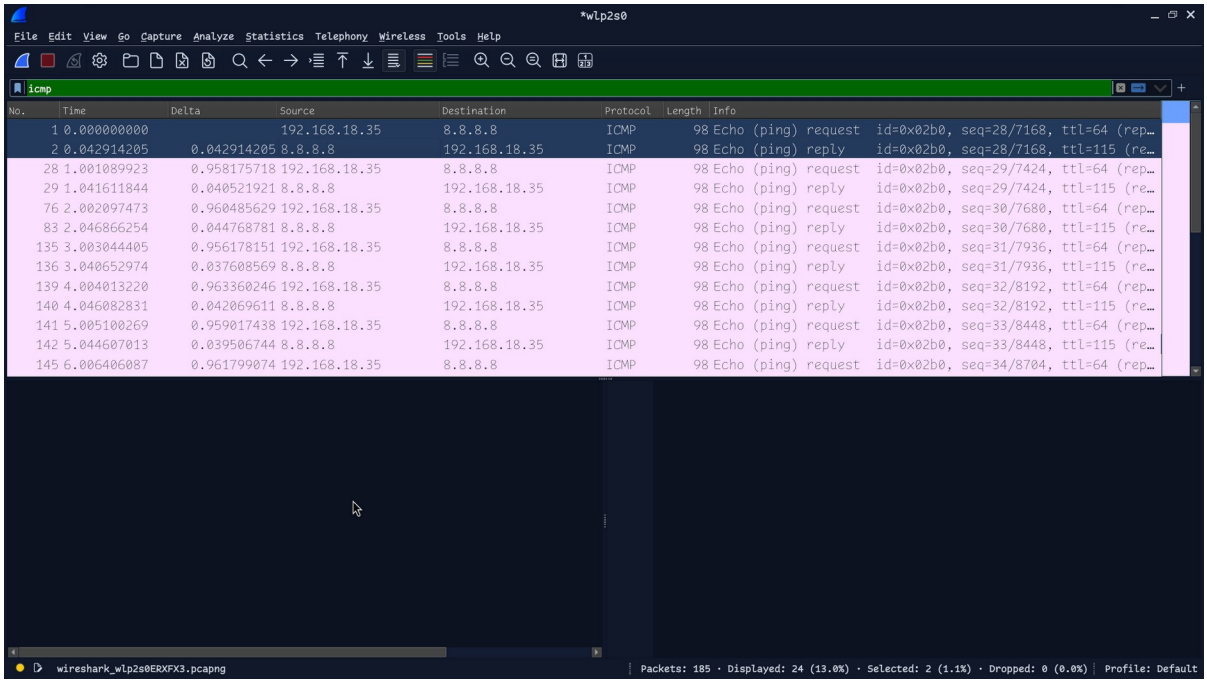
You should see:

- Echo Request
- Echo Reply
- TTL
- Sequence number
- Checksums

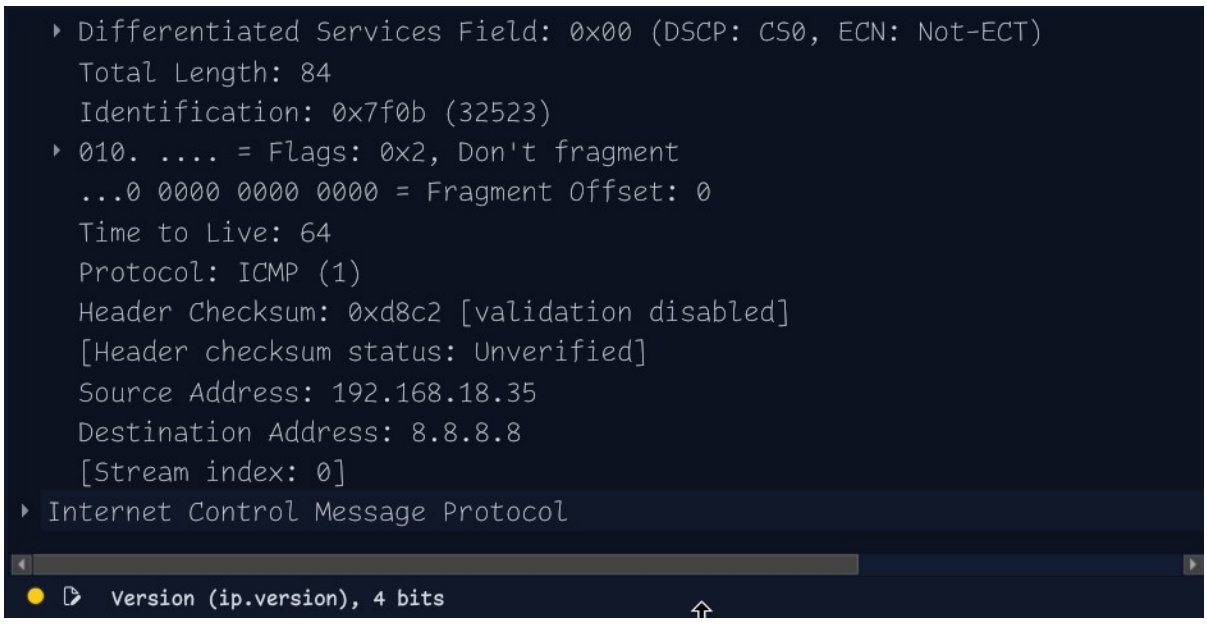
Task 1

1. Capture ICMP packets and **take screenshots** of:

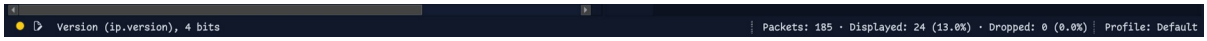
   a. Packet list



   b. Packet details



c. Packet bytes



2. Identify:

- Source IP: 192.168.18.35
- Destination IP: 8.8.8.8
- TTL values: 64
- ICMP type: 8
- ICMP code: 0

Experiment 2 – Capturing HTTP & HTTPS

Traffic Theory

HTTP is plaintext (visible), while HTTPS is encrypted (hidden payload). This experiment helps students understand encrypted vs unencrypted communication.

Steps

Step 1 — Apply HTTP Filter

http

Step 2 — Generate Traffic

Open browser → visit a test HTTP site:

http://example.com

Step 3 – Inspect HTTP Headers

Look for:

- GET request
- Status codes
- Host field
- User-Agent

- Server details

Step 4 — Inspect HTTPS

Use filter:

tls

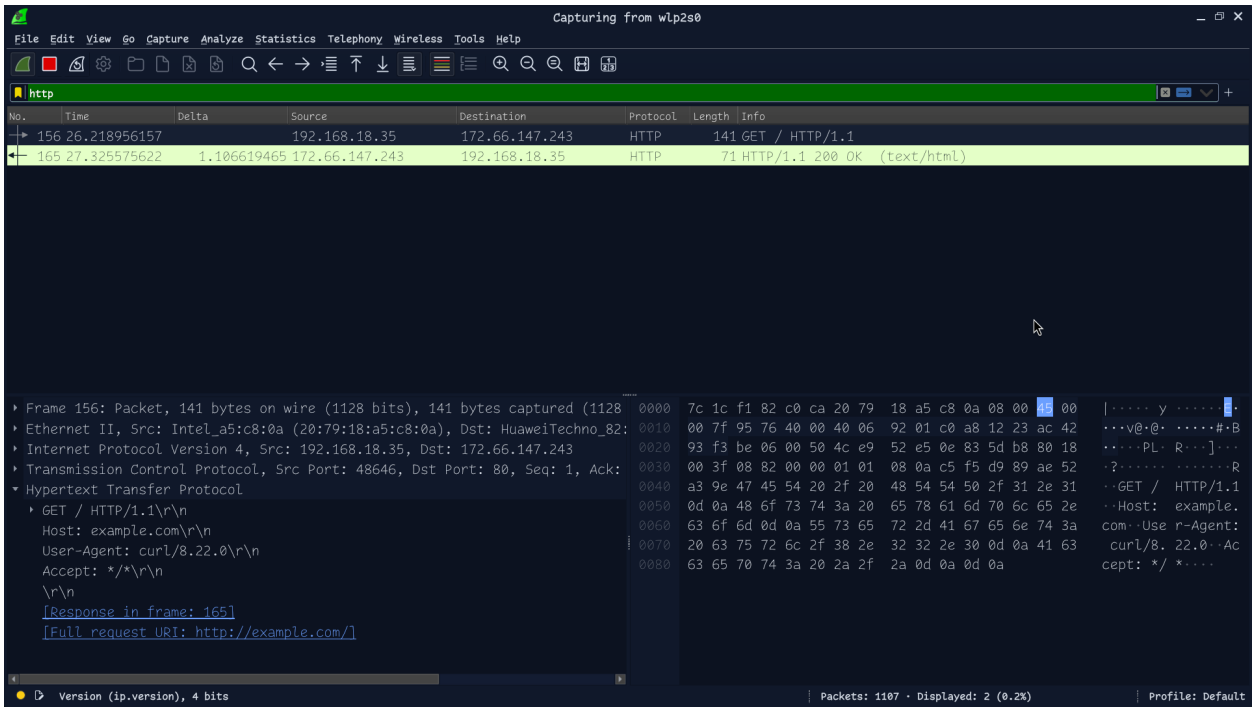
Observe:

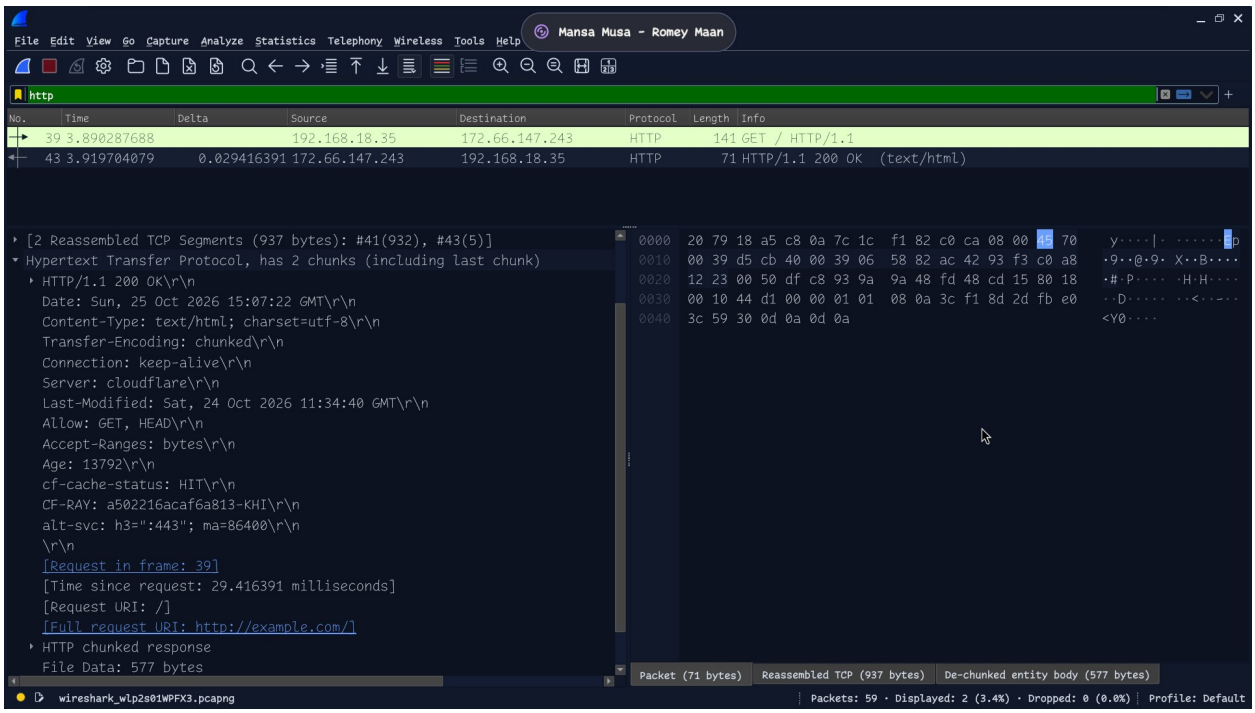
- Server Hello
- Certificate exchange
- Encrypted Application Data

Task 2

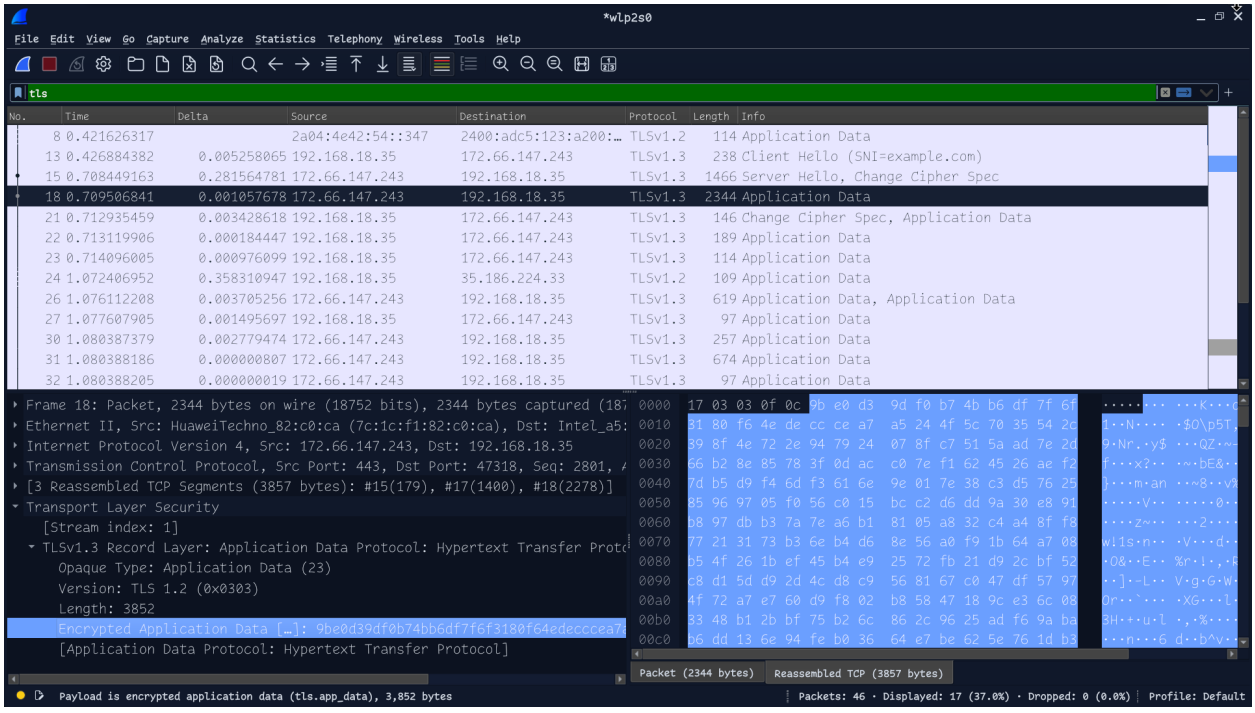
Capture **both HTTP & HTTPS packets.**

HTTP




File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Mansa Musa - Romey Maan
http
No. Time Delta Source Destination Protocol Length Info
39 3.890287688 192.168.18.35 172.66.147.243 HTTP 141 GET / HTTP/1.1
43 3.919704079 0.029416391 172.66.147.243 192.168.18.35 HTTP 71 HTTP/1.1 200 OK (text/html)
[2 Reassembled TCP Segments (937 bytes): #41(932), #43(5)]
Hypertext Transfer Protocol, has 2 chunks (including last chunk)
HTTP/1.1 200 OK\r\n
Date: Sun, 25 Oct 2026 15:07:22 GMT\r\n
Content-Type: text/html; charset=utf-8\r\n
Transfer-Encoding: chunked\r\n
Connection: keep-alive\r\n
Server: cloudflare\r\n
Last-Modified: Sat, 24 Oct 2026 11:34:40 GMT\r\n
Allow: GET, HEAD\r\n
Accept-Ranges: bytes\r\n
Age: 13792\r\n
cf-cache-status: HIT\r\n
CF-RAY: a502216acaf6a813-KHI\r\n
alt-svc: h3=":443"; ma=86400\r\n
\r\n
[Request in frame: 39]
[Time since request: 29.416391 milliseconds]
[Request URI: /]
[Full request URI: http://example.com/]
HTTP chunked response
File Data: 577 bytes
0000 20 79 18 a5 c8 0a 7c 1c f1 82 c0 ca 08 00 45 70
0010 00 39 d5 cb 40 00 39 06 58 82 ac 42 93 f3 c0 a8
0020 12 23 00 50 df c8 93 9a 9a 48 fd 48 cd 15 80 18
0030 00 10 44 d1 00 00 01 01 08 0a 3c f1 8d 2d fb e0
0040 3c 59 30 0d 0a 0d 0a
Packet (71 bytes)
Reassembled TCP (937 bytes)
De-chunked entity body (577 bytes)
wireshark_wlp2s01WPFX3.pcapng
Packets: 59 · Displayed: 2 (3.4%) · Dropped: 0 (0.0%)
Profile: Default

HTTPS


*wlp2s0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tls
No. Time Delta Source Destination Protocol Length Info
8 0.421626317 2a04:4e42:54::347 2400:adc5:123:a200:… TLSv1.2 114 Application Data
13 0.426884382 0.005258065 192.168.18.35 172.66.147.243 TLSv1.3 238 Client Hello (SNI=example.com)
15 0.708449163 0.281564781 172.66.147.243 192.168.18.35 TLSv1.3 1466 Server Hello, Change Cipher Spec
18 0.709506841 0.001057678 172.66.147.243 192.168.18.35 TLSv1.3 2344 Application Data
21 0.712935459 0.003428618 192.168.18.35 172.66.147.243 TLSv1.3 146 Change Cipher Spec, Application Data
22 0.713119906 0.000184447 192.168.18.35 172.66.147.243 TLSv1.3 189 Application Data
23 0.714096005 0.000976099 192.168.18.35 172.66.147.243 TLSv1.3 114 Application Data
24 1.072406952 0.358310947 192.168.18.35 35.186.224.33 TLSv1.2 109 Application Data
26 1.076112208 0.003705256 172.66.147.243 192.168.18.35 TLSv1.3 619 Application Data, Application Data
27 1.077607905 0.001495697 192.168.18.35 172.66.147.243 TLSv1.3 97 Application Data
30 1.080387379 0.002779474 172.66.147.243 192.168.18.35 TLSv1.3 257 Application Data
31 1.080388186 0.000000807 172.66.147.243 192.168.18.35 TLSv1.3 674 Application Data
32 1.080388205 0.000000019 172.66.147.243 192.168.18.35 TLSv1.3 97 Application Data
Frame 18: Packet, 2344 bytes on wire (18752 bits), 2344 bytes captured (187
Ethernet II, Src: HuaweiTechno_82:c0:ca (7c:1c:f1:82:c0:ca), Dst: Intel_a5:
Internet Protocol Version 4, Src: 172.66.147.243, Dst: 192.168.18.35
Transmission Control Protocol, Src Port: 443, Dst Port: 47318, Seq: 2801, A
[3 Reassembled TCP Segments (3857 bytes): #15(179), #17(1400), #18(2278)]
Transport Layer Security
[Stream index: 1]
TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Proto
Opaque Type: Application Data (23)
Version: TLS 1.2 (0x0303)
Length: 3852
Encrypted Application Data […]: 9be0d39df0b74bb6df7f6f3180f64edecccea7
[Application Data Protocol: Hypertext Transfer Protocol]
0000 17 03 03 0f 0c 9b e0 d3 9d f0 b7 4b b6 df 7f 6f
0010 31 80 f6 4e de cc ce a7 a5 24 4f 5c 70 35 54 2c
0020 39 8f 4e 72 2e 94 79 24 07 8f c7 51 5a ad 7e 2d
0030 66 b2 8e 85 78 3f 0d ac c0 7e f1 62 45 26 ae f2
0040 7d b5 d9 f4 6d f3 61 6e 9e 01 7e 38 c3 d5 76 25
0050 85 96 97 05 f0 56 c0 15 bc c2 d6 dd 9a 30 e8 91
0060 b8 97 db b3 7a 7e a6 b1 81 05 a8 32 c4 a4 8f f8
0070 77 21 31 73 b3 6e b4 d6 8e 56 a0 f9 1b 64 a7 08
0080 b5 4f 26 1b ef 45 b4 e9 25 72 fb 21 d9 2c bf 52
0090 c8 d1 5d d9 2d 4c d8 c9 56 81 67 c0 47 df 57 97
00a0 4f 72 a7 e7 60 d9 f8 02 b8 58 47 18 9c e3 6c 08
00b0 33 48 b1 2b bf 75 b2 6c 86 2c 96 25 ad f6 9a ba
00c0 b6 dd 13 6e 94 fe b0 36 64 e7 be 62 5e 76 1d b3
Packet (2344 bytes)
Reassembled TCP (3857 bytes)
Payload is encrypted application data (tls.app_data), 3,852 bytes
Packets: 46 · Displayed: 17 (37.0%) · Dropped: 0 (0.0%)
Profile: Default

□ Which fields are visible in HTTP but hidden in HTTPS?

- **URL path: The specific page/file path (e.g., `/login`).**
- **Headers: Cookies, authorization tokens, and user details.**
- **Payload/Body: Passwords, form inputs, HTML, and responses.**
- **Status codes: Server replies like `200 OK` or `404`.*(Note: IP addresses and port numbers remain visible in both)*.**

□ Why is HTTPS considered secure?

- **Encryption: Scrambles data into unreadable text so sniffers can't read it.**
- **Integrity: Prevents data from being altered or injected in transit.**
- **Authentication: Uses SSL/TLS certificates to verify the website is real, not an imposter.**

Experiment 3 – Analyzing TCP 3-Way Handshake Theory

TCP connection establishment:

1. SYN
2. SYN-ACK
3. ACK

Step 1 — Apply Filter

tcp.flags.syn==1

Step 2 — Initiate Connection

Run:

curl http://example.com

Step 3 — Identify 3 Packets

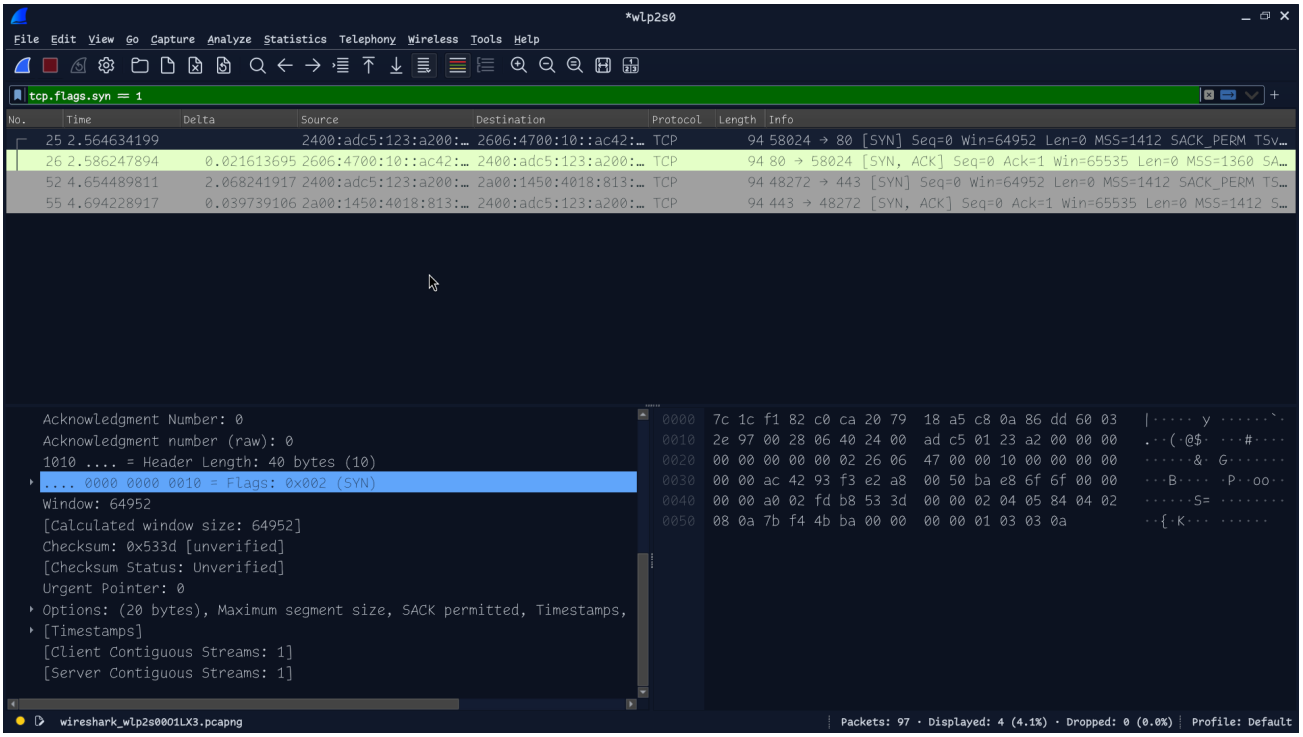
Find:

1. SYN (Client → Server)
2. SYN-ACK (Server → Client)
3. ACK (Client → Server)

Step 4 – Inspect Fields

Inside packet details expand:

- TCP flags
- Sequence number
- Acknowledgment number
- Window size
- MSS value



Experiment 4 – Using Display & Capture

Filters Difference

- **Display filters:** After capturing
- **Capture filters:** Before capturing

Display Filters

Use the following:

Filter	Purpose
icmp	ICMP packets
arp	ARP requests/responses
dns	DNS

queries tcp.port==80

HTTP-traffic

ip.addr == x.x.x.x Filter by a

specific IP Try:

dns

http

tcp.flags.reset=1

Capture Filters

Before capture → **Capture** → **Options** → **Capture Filter**

Examples:

tcp port 80

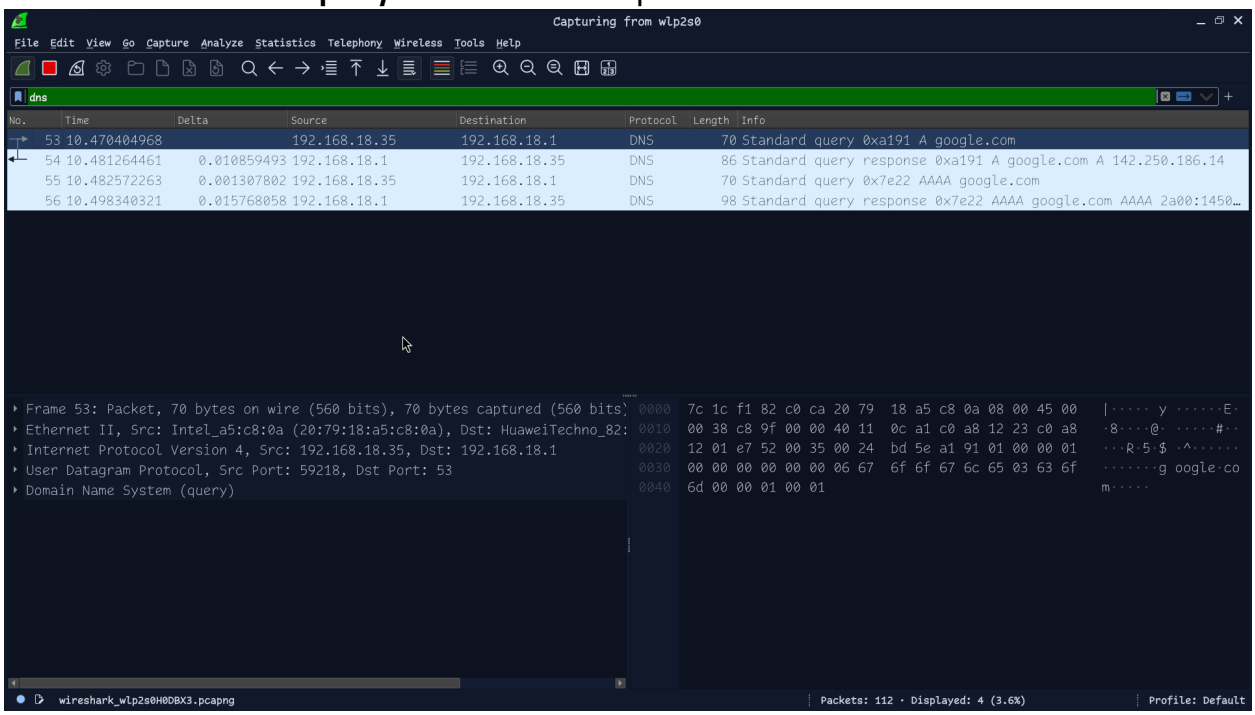
icmp

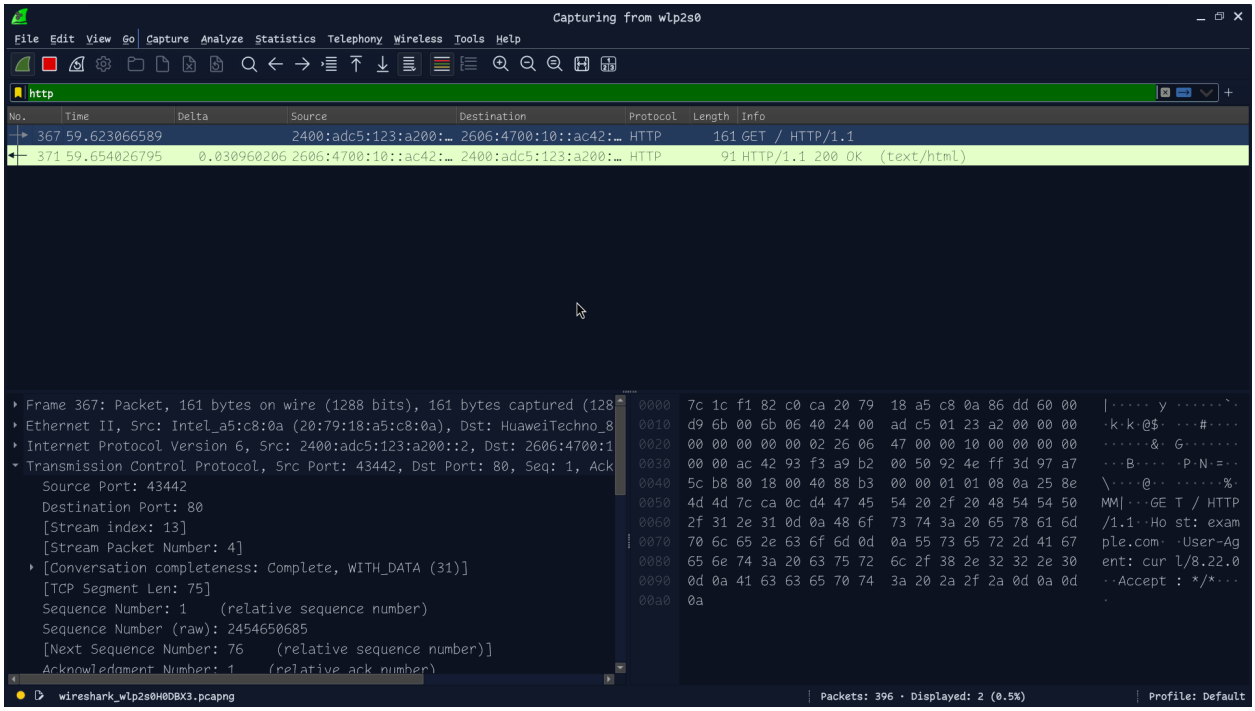
host 8.8.8.8

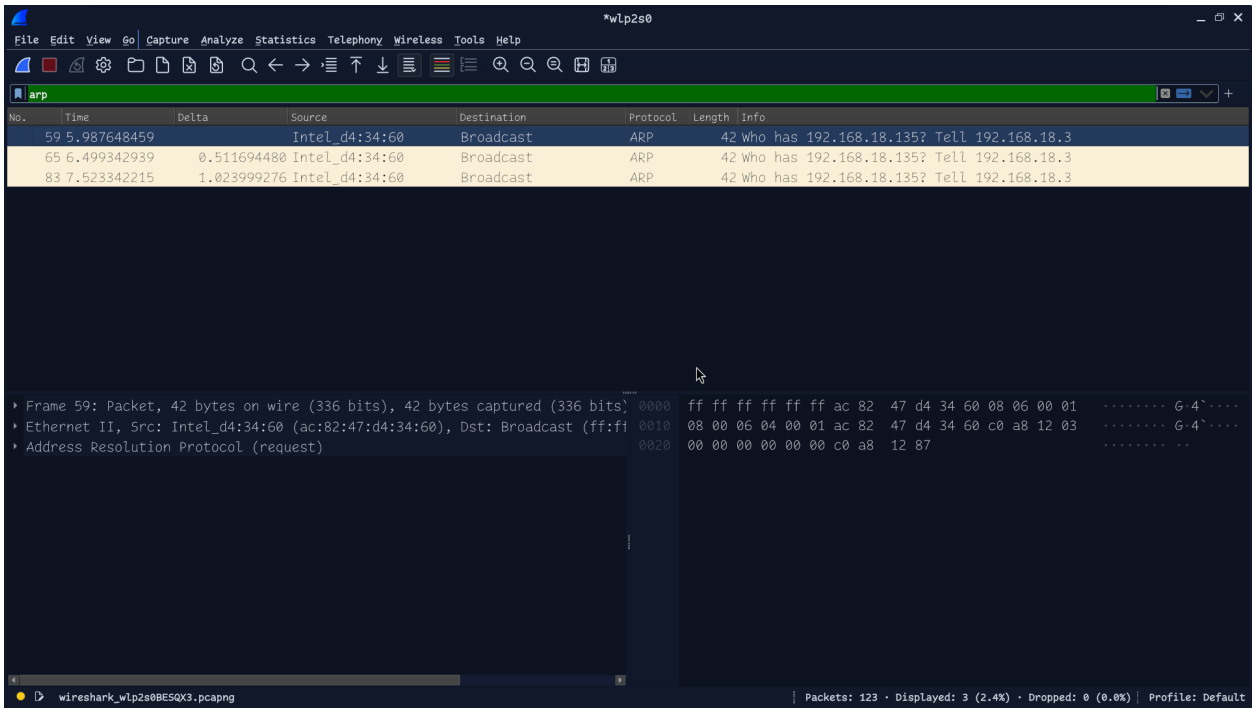
net 192.168.10.0/24

Task 4

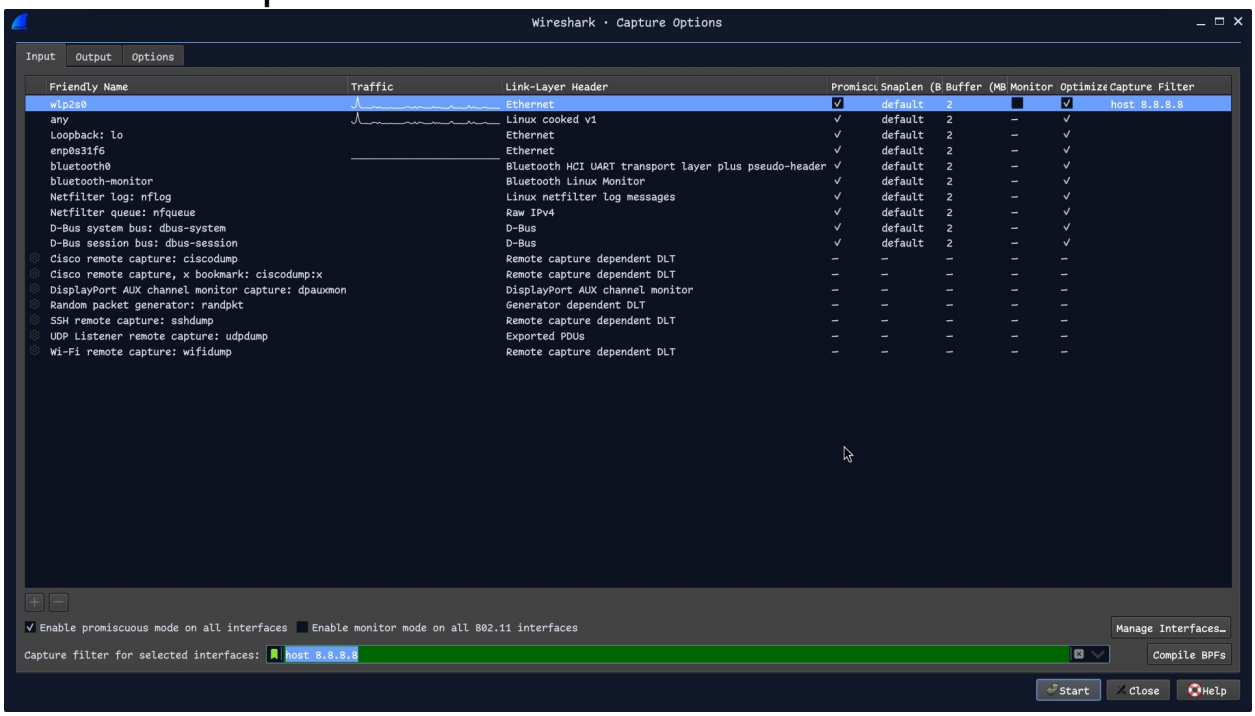
1. Use **three display filters** and provide screenshots.

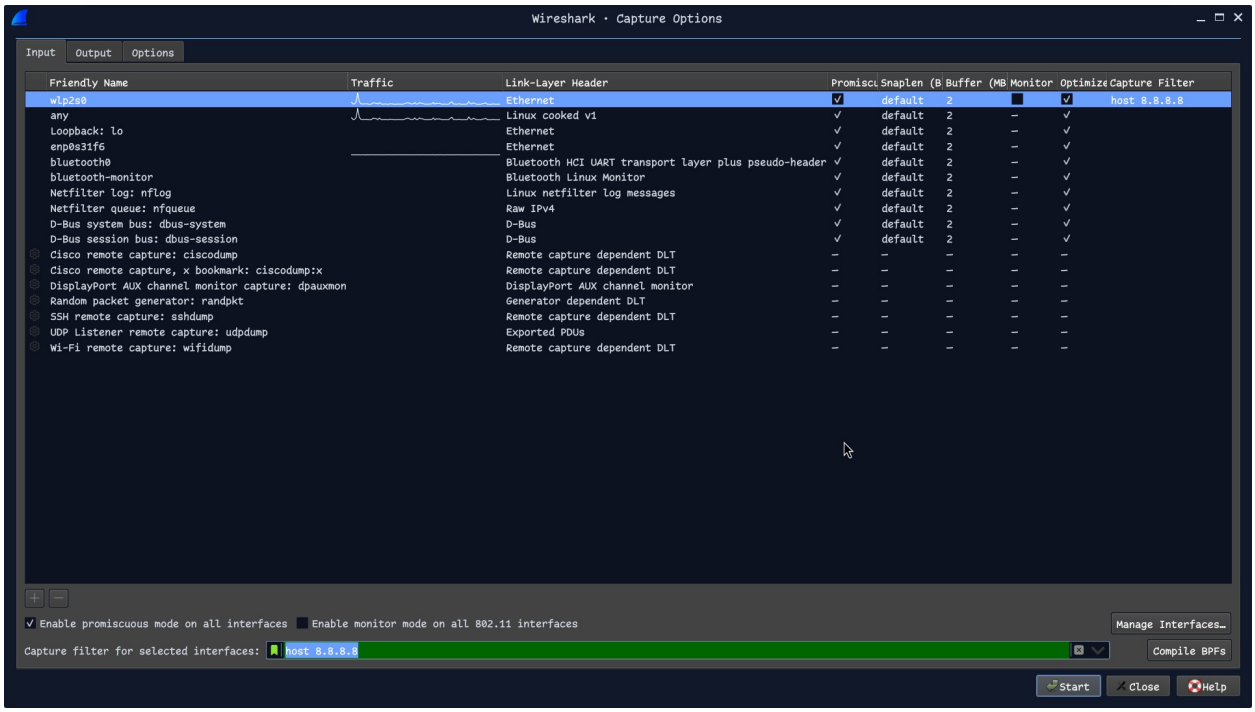



Capturing from wlp2s0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http
No. Time Delta Source Destination Protocol Length Info
367 59.623066589 2400:adc5:123:a200:… 2606:4700:10::ac42:… HTTP 161 GET / HTTP/1.1
371 59.654026795 0.030960206 2606:4700:10::ac42:… 2400:adc5:123:a200:… HTTP 91 HTTP/1.1 200 OK (text/html)
Frame 367: Packet, 161 bytes on wire (1288 bits), 161 bytes captured (128
Ethernet II, Src: Intel_a5:c8:0a (20:79:18:a5:c8:0a), Dst: HuaweiTechno_8
Internet Protocol Version 6, Src: 2400:adc5:123:a200::2, Dst: 2606:4700:1
Transmission Control Protocol, Src Port: 43442, Dst Port: 80, Seq: 1, Ack
Source Port: 43442
Destination Port: 80
[Stream index: 13]
[Stream Packet Number: 4]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 75]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 2454650685
[Next Sequence Number: 76 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
0000 7c 1c f1 82 c0 ca 20 79 18 a5 c8 0a 86 dd 60 00
0010 d9 6b 00 6b 06 40 24 00 ad c5 01 23 a2 00 00 00
0020 00 00 00 00 00 02 26 06 47 00 00 10 00 00 00 00
0030 00 00 ac 42 93 f3 a9 b2 00 50 92 4e ff 3d 97 a7
0040 5c b8 80 18 00 40 88 b3 00 00 01 01 08 0a 25 8e
0050 4d 4d 7c ca 0c d4 47 45 54 20 2f 20 48 54 54 50
0060 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 65 78 61 6d
0070 70 6c 65 2e 63 6f 6d 0d 0a 55 73 65 72 2d 41 67
0080 65 6e 74 3a 20 63 75 72 6c 2f 38 2e 32 32 2e 30
0090 0d 0a 41 63 63 65 70 74 3a 20 2a 2f 2a 0d 0a 0d
00a0 0a
wireshark_wlp2s0H0DBX3.pcapng
Packets: 396 · Displayed: 2 (0.5%)
Profile: Default


*wlp2s0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
arp
No. Time Delta Source Destination Protocol Length Info
59 5.987648459 Intel_d4:34:60 Broadcast ARP 42 Who has 192.168.18.135? Tell 192.168.18.3
65 6.499342939 0.511694480 Intel_d4:34:60 Broadcast ARP 42 Who has 192.168.18.135? Tell 192.168.18.3
83 7.523342215 1.023999276 Intel_d4:34:60 Broadcast ARP 42 Who has 192.168.18.135? Tell 192.168.18.3
Frame 59: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits)
Ethernet II, Src: Intel_d4:34:60 (ac:82:47:d4:34:60), Dst: Broadcast (ff:ff
Address Resolution Protocol (request)
0000 ff ff ff ff ff ff ac 82 47 d4 34 60 08 06 00 01
0010 08 00 06 04 00 01 ac 82 47 d4 34 60 c0 a8 12 03
0020 00 00 00 00 00 00 c0 a8 12 87
wireshark_wlp2s0BESQX3.pcapng
Packets: 123 · Displayed: 3 (2.4%) · Dropped: 0 (0.0%)
Profile: Default

2. Use **two capture filters** and describe the results.





3. Explain why capture filters are faster for large networks.
   Kernel vs. User Space:

   - Capture filters run directly in the Linux kernel using BPF (Berkeley Packet Filter). Packets that don't match the rule are dropped immediately before reaching application memory.

   - Display filters force Wireshark to copy *every single packet* from kernel space to user space, write it to disk or RAM, and decode every protocol header before deciding whether to show it.

   Resource Usage:

   - On high-speed, busy networks (gigabit/multi-gigabit links), processing all packets in user space causes high CPU usage, packet drops, and GUI crashes. Dropping unwanted traffic at the kernel level keeps memory usage low and prevents dropped packets.

Experiment 5 – Permissions Configuration

(LinuxOnly) Steps

Step 1 — Add User to Wireshark

Group sudo usermod -aG wireshark

$USER sudo reboot

Step 2 — Verify

Open Wireshark **without root.**

Task 5

Write:

- **Why Wireshark should NOT run as root.**

► Violation of Least Privilege: Packet capturing requires root-level network access, but dissecting packets, decoding protocols, and rendering the GUI do not. Running the entire program as root grants unnecessary superuser privileges to millions of lines of complex code.

► Separation of Concerns: Wireshark isolates the capturing task into a small, purpose-built CLI tool called dumpcap. By giving Linux capabilities (CAP_NET_RAW and CAP_NET_ADMIN) only to dumpcap, normal users can capture packets while the main GUI runs unprivileged.

- **What are the risks?**

► Remote Code Execution (RCE): Wireshark has hundreds of dissectors written in C/C++ to parse complex protocols. In the past, attackers have found memory corruption bugs (like buffer overflows or use-after-free) in these dissectors. If you run Wireshark as root, an attacker only has to send a crafted, malformed packet over your Wi-Fi or LAN to execute code with full root privileges.

► Total System Compromise: An exploit running as a standard user is contained within user space, but an exploit running under root gives attackers unrestricted control over your filesystem, kernel, user accounts, and system configuration.

► GUI & Dependency Exposure: Large graphical frameworks (like Qt) and plugins are not built or audited to run securely with root permissions, introducing additional security holes.

Class Participation Task (10 Marks)

Report: Network Packets and Traffic Security

1. Comparison of ICMP, HTTP, ARP, and DNS Packets

Protocol	Purpose	Information Revealed
ICMP	Checks connectivity and reports network errors.	IP addresses and network structure.
HTTP	Transfers web pages and data between clients and servers.	URLs, webpage content, and login credentials if transmitted without encryption.
ARP	Maps IP addresses to MAC addresses on a local network.	Device IP and MAC addresses.
DNS	Converts domain names into IP addresses.	Websites and domains a user accesses.

2. Packets That Reveal Sensitive Information

- **HTTP: Reveals the most sensitive information when unencrypted, including passwords, session cookies, and messages.**
- **DNS: May reveal browsing activity because domain queries are often unencrypted.**
- **ARP: Exposes local network devices and their IP-to-MAC address mappings.**
- **ICMP: Can reveal IP addresses, network paths, and information useful for network reconnaissance.**

3. Three Ways to Protect Network Traffic

1. **Use Encryption: Use HTTPS, TLS, and encrypted DNS to protect data from being read by unauthorized parties.**
2. **Use a VPN: Encrypt network traffic between your device and the**

VPN server, reducing exposure on local networks.

3. Use Firewalls: Filter unwanted traffic, restrict unnecessary connections, and block unauthorized access.

Conclusion

Network packets serve different purposes, but they can expose sensitive information when traffic is unencrypted or improperly controlled. Encryption, VPNs, and firewalls help improve network privacy and security.